

MLOps on Cloudera AI

From experiment to governed production endpoint —
without moving your data



Live demonstration · Credit risk model · Cloudera AI Workbench

Models get built. Far fewer reach production.

The blocker is rarely the algorithm. It is everything around it — the toolchain, the evidence trail, and the promotion path.



A fragmented toolchain

Notebooks in one place, a tracking server server in another, serving owned by a third a third team. Data gets copied to all three three — and that is where governance governance breaks.



No usable evidence trail

Six months on, nobody can say which run produced the model that declined an application, or what data it saw.



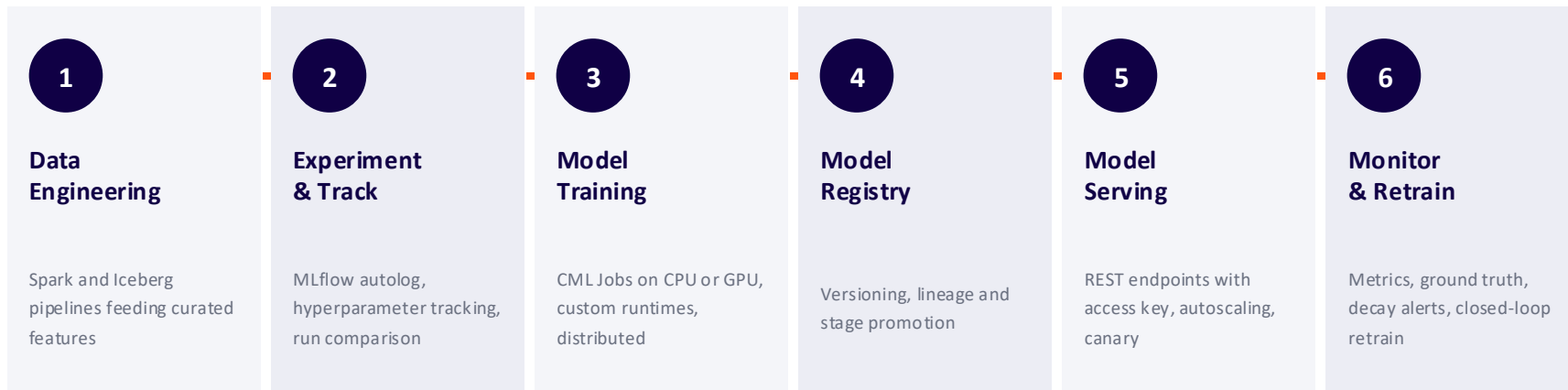
Promotion by hand

Model quality is a convention enforced in review meetings, not a control enforced in the pipeline. Retraining becomes a project each time.

Cloudera AI closes all three by keeping the lifecycle where the data already lives.

One lifecycle, one platform, one security model

Each stage below is a native Cloudera AI capability — not an integration you assemble and maintain.



Underneath all six: **Cloudera SDX** — Ranger RBAC, Atlas lineage, enterprise auth, encryption, audit logging and a shared data catalog, on-premises and in cloud alike.

Three services, one console



Cloudera AI Workbench

Where models are built and iterated. Projects, sessions, jobs, experiments and project-scoped model deployments — with runtimes and compute chosen per workload.



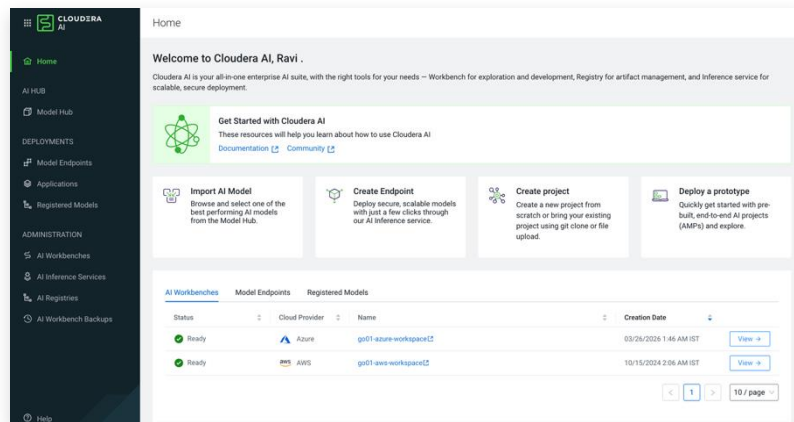
Cloudera AI Registry

Where a model becomes a managed asset. Versions, parameters, metrics, artifacts, metadata and lineage — the record your model risk function needs.



Cloudera AI Inference

Production-grade serving for predictive and generative models, with autoscaling, canary rollout and endpoint metrics.



All three are reached from a single Cloudera AI console.

The demo

Credit risk, end to end

Eight steps, roughly thirty minutes, all inside one project.



Will this borrower default?

A binary classifier over eight loan-application features — the shape of a real BFSI credit-decisioning model, small enough to train live.

FEATURES USED

loan_amount

Requested amount

credit_score

FICO-style, 300–850

debt_to_income

Debt service ratio

num_delinquencies

Past delinquencies

annual_income

Applicant income

employment_years

Tenure in current role

num_credit_lines

Open credit lines

loan_purpose

Home, auto, education...

10,000

synthetic loan records

XGBoost

gradient-boosted classifier

0.72 / 0.50

ROC-AUC and F1 promotion floor

The dataset is synthetic so the demo is self-contained and reproducible. In your environment step one becomes a Spark, Hive or Iceberg read against curated tables — every stage after it is unchanged.

The whole use case lives in one project

- **Isolated by design.** Code, files, libraries, environment variables and access control, scoped to a single use case.
- **Git-native.** Projects can be created from a Git repository, so the demo you are watching is version-controlled source, not a snapshot.
- **Everything in view.** Models, jobs and files on one page — the lifecycle is legible to someone who has never opened the project.

The screenshot displays the Cloudera AI Workbench interface for a project named 'MLOPS Workshop'. The left sidebar contains a navigation menu with options like Home, All Projects, PROJECT Overview, AI Studios, Sessions, Data, Experiments, Model Deployments, Jobs, Applications, Files, Collaborators, Project Settings, AMPs, Runtime Catalog, Learning Hub, and Help. The main content area is divided into three sections: Models, Jobs, and Files.

Models Section:

Model Deployment	Source	Status	Replicas	CPU	Memory	Last Deployed	Actions
Workshop - Ravi MLOPS	cmLm...	Deployed	0 / 1	0	0 GiB	May 29, 2026, 06:24 PM	Start

Jobs Section:

Name	Runs / Failures	Duration	Status	Latest Run	Actions
Validate Model KPIs	2 / 0	00:08	Success	3/24/26 11:29 AM	Run
Train Credit Risk Model	2 / 0	00:39	Success	3/24/26 11:28 AM	Run
Generate Loan Data	2 / 0	00:02	Success	3/24/26 11:28 AM	Run

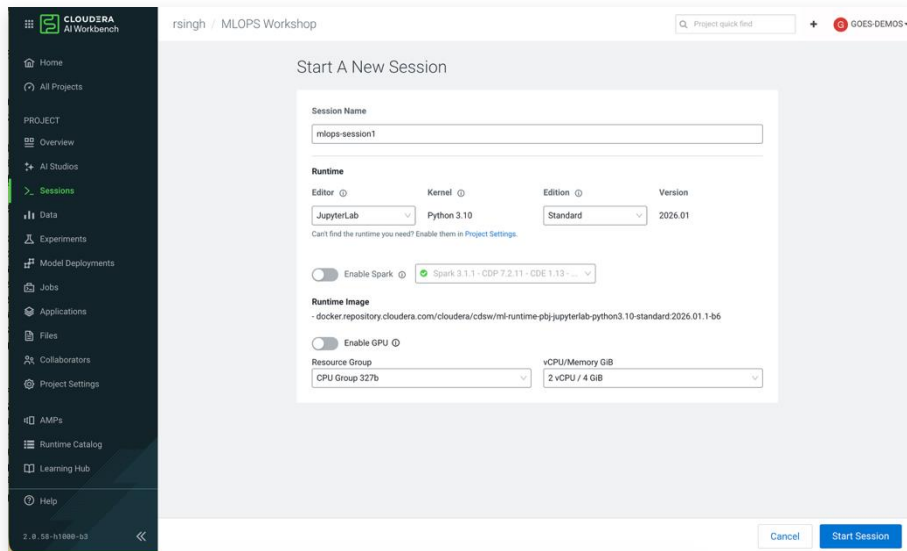
Files Section:

Name	Size	Last Modified	Actions
01_generate_data.py	1.28 kiB	3/22/26 12:16 PM	Edit
02_train_model.py	4.48 kiB	3/22/26 12:16 PM	Edit
03_predict.py	4.00 kiB	3/22/26 12:16 PM	Edit
04_test_app.py	1.91 kiB	3/22/26 12:16 PM	Edit
05_validate_model.py	2.99 kiB	3/22/26 12:16 PM	Edit

Why it matters When a use case is a project rather than a folder on somebody's laptop, onboarding a new data scientist takes minutes and offboarding one costs nothing.

Choose the environment, not the machine

- **Runtime, not install.** Editor, kernel, edition and version are selected per session — JupyterLab or Workbench, CPU or GPU.
- **Right-sized compute.** Resource profile is picked at launch and released on stop, so nobody holds a large instance overnight.
- **Spark on a toggle.** Distributed processing against Data Lake tables is a switch, not a separate cluster request.



Why it matters Data scientists stop waiting on infrastructure tickets, and platform teams stop paying for idle notebook servers.

Training is a workload, not a laptop script

- **Deterministic.** Seeded data generation and training — the same inputs give the same model, every run.
- **Tracked without effort.** The script contains no tracking URI, no server address and no credentials. Cloudera AI sets them in every session.
- **Portable.** The identical script runs in a session, as a scheduled Job, and in an external CI pipeline — unchanged.

The screenshot shows the Cloudera AI Workbench interface. On the left is a file explorer with a tree view containing files like `01_generate_data.py`, `02_train_model.py`, `03_predict.py`, `04_test_api.py`, `05_validate_model.py`, `cdsw-build.sh`, `cnl_model.py`, `credit_risk_model.py`, `label_encoder.pkl`, `loan_data.csv`, `README.md`, `requirements.txt`, and `Untitled.ipynb` (highlighted in blue). The main area is a terminal window titled 'Terminal 1' showing the execution of a Python script. The script generates a dataset and trains a model. The output includes a table of metrics:

	precision	recall	f1-score	support
0	0.71	0.77	0.74	1220
1	0.59	0.52	0.55	708

Additional metrics shown are accuracy (0.65), macro avg (0.64), and weighted avg (0.67). The ROC-AUC is 0.7480. The terminal also shows warnings about integer columns in the dataset and the binding of ports 8090 and 8088.

Watch for The run ID printed at the end is the thread that ties this console output to the experiment record, the artifact, and eventually the deployed endpoint.

Every run is captured, automatically

- **Nothing is optional.** Hyperparameters, dataset statistics, KPIs, the serialised model, its signature and the encoder — all recorded per run.
- **Open by design.** This is standard MLflow, running as a managed service. Your team keeps the API it already knows; no proprietary format to migrate out of later.
- **Governed.** Runs inherit project access control, so tracking is not a second security perimeter to manage.

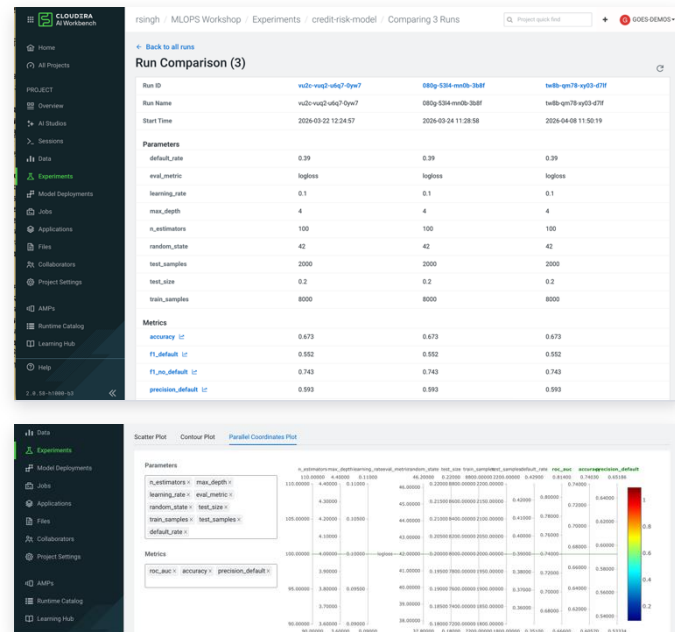
The top screenshot shows the 'Experiments' view in the Cloudera AI Workbench. It displays a table of experiments with columns: Name, Creator, Created At, and Last Updated. One experiment, 'credit-risk-model', is highlighted. The bottom screenshot shows the 'Experiment' details view for 'credit-risk-model'. It displays the experiment name, ID, artifact location, and a list of runs. Below the runs list, there is a table of metrics for each run.

Stat	Start Time	Run Name	Duration	User	Source	Version	Models	Parameters	Metrics	Tags		
✓	2028-03-22 12:12	vc2b...	17.8s	rsingh	25cc85	0.39	logloss	0.1	0.673	0.5521	0.7425	y0y11...
✓	2028-03-22 12:12	q5e6...	25.8s	rsingh	25cc85	0.39	logloss	0.1	0.673	0.5521	0.7425	r3rfr...
✓	2028-03-24 11:11	080g...	20.5s	rsingh	25cc85	0.39	logloss	0.1	0.673	0.5521	0.7425	q7w7L...
✓	2028-04-08 11:11	tw8b...	20.1s	rsingh	25cc85	0.39	logloss	0.1	0.673	0.5521	0.7425	7wi22...

Why it matters This is the answer to "which run produced the model in production?" — asked by an auditor, not by a data scientist.

Pick the winner on evidence

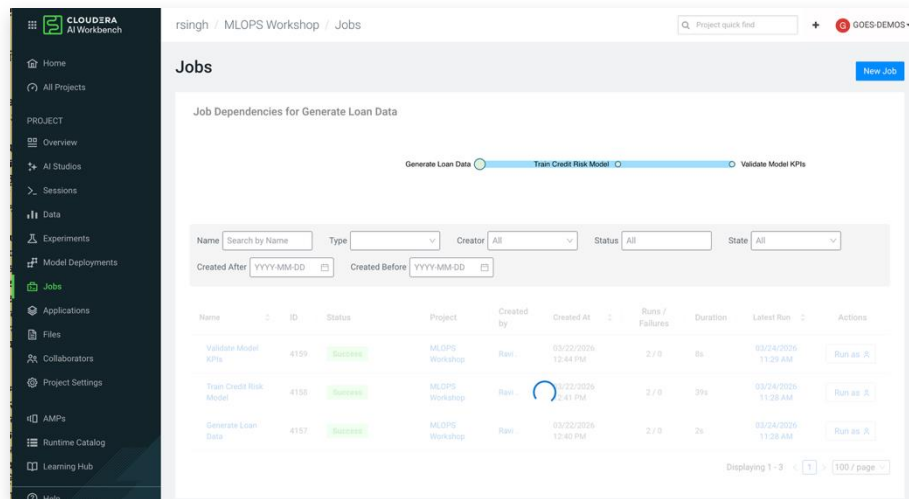
- **Side by side.** Parameters and metrics diffed across selected runs, so a hyperparameter change and its KPI effect sit on one screen.
- **Visual exploration.** Scatter, contour and parallel-coordinates views make the shape of the search space obvious rather than tabular.
- **Reproducible selection.** The chosen run carries its artifacts with it, so promoting it does not mean rebuilding it.



Why it matters Model selection stops being a conversation and becomes a record — one that survives the departure of the person who made the call.

Quality as a control, not a convention

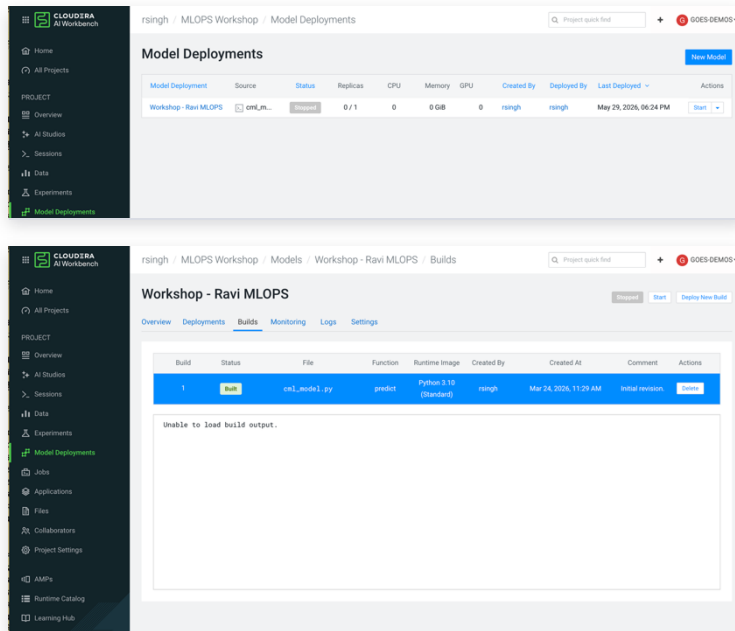
- **A gate in code.** Validation exits non-zero when ROC-AUC or F1 on the default class falls below the agreed floor — the job turns red and the chain stops.
- **Chained pipeline.** Generate, train and validate run as dependent Jobs on a schedule, each with its own runtime, sizing and alerting.
- **The right metric.** The floor is set on the minority class the business cares about — at a 39% default rate, accuracy alone would flatter a useless model.



Why it matters Your model risk function sets the numbers. The platform enforces them on every retrain, without anyone having to remember to check.

A Python function becomes a governed endpoint

- **No serving stack to build.** Point at a file and a function. Cludera AI containerises it, exposes it over HTTPS and keeps it running without a session.
- **Authenticated by default.** Every request carries an access key, and the endpoint inherits the platform identity and access model.
- **Production isolated.** The project is snapshotted at deploy time, so later development changes cannot disturb a running production model.



Why it matters No Dockerfile, no ingress rules, no Kubernetes work — and no separate serving platform on the bill.

Retrain, rebuild, redeploy

- **Same pipeline.** Retraining is the jobs you already saw — a new tracked run, the gate re-evaluated, nothing bespoke.
- **Zero-downtime refresh.** A rebuild picks up the new artifacts and cuts traffic over; replicas and sizing are configuration, not a rewrite.
- **Traceable lineage.** Every deployment points back to a build, and every build back to a training run.

The screenshot shows the Cloudera AI Workbench interface. The left sidebar contains navigation options: Home, All Projects, PROJECT, Overview, AI Studios, Sessions, Data, Experiments, **Model Deployments** (highlighted), Jobs, Applications, Files, Collaborators, Project Settings, AMPs, Runtime Catalog, and Learning Hub.

The main content area is titled 'Workshop - Ravi MLOPS' and shows the 'Deployments' tab. It contains a table with the following data:

Id	Build	Status	Deployed At	Stopped At	Deployed By
4488	1	Stopped	Never	Jun 4, 2026, 11:50 PM	rsingh
4489	1	Stopped	Never	Apr 10, 2026, 08:31 PM	rsingh

Below the table, the details for the selected deployment (Id: 4488) are shown:

- Model:**
 - Id: 2488
 - Name: Workshop - Ravi MLOPS
 - Description: credit-risk-model Workshop - Ravi MLOPS
- Build:**
 - Build Number: 1
 - UUID: e414bbfb-2219-4b86-8df3-773e2c54632d
 - File: cn1_model.py
 - Function: predict
 - Runtime: Python 3.10 (Standard)
- Deployment:**
 - Id: 4488
 - Deployed At: Never
 - Last Restarted: Never
 - Last Updated: Jul 30, 2026, 05:23 AM
- Resources:**
 - Replicas: 1
 - Total CPU: 2 vCPUs
 - Total Memory: 4.00 GiB

Why it matters The loop your model risk policy already requires — running as infrastructure rather than as a quarterly manual project.

Closing the loop on a live model

Deployment is the start of the lifecycle, not the end. Cloudera AI records what a model did in production and lets you compare it with what actually happened.

CAPTURED NATIVELY

■ Prediction-time metrics

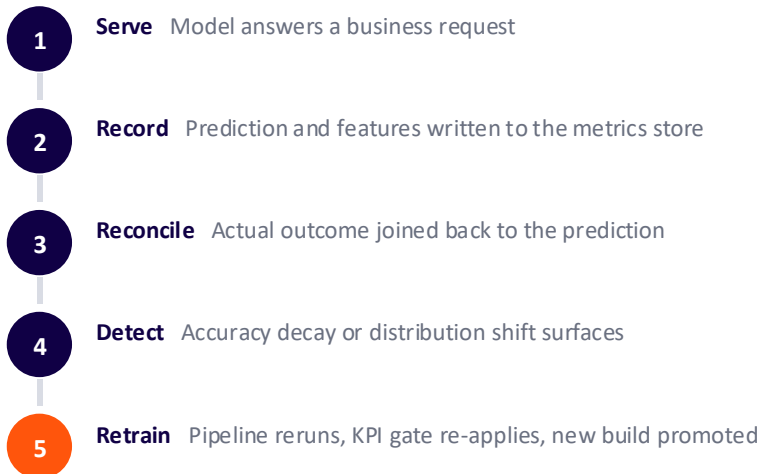
`track_metric()` records feature values and predictions per call, each with its own identifier.

■ Ground truth, later

`track_delayed_metric()` writes the real outcome back against the original prediction when it arrives.

■ Service health

Success rate, requests per second and request counts per replica, on the Monitoring tab.



Be precise The metrics store, ground-truth tracking and retraining pipeline are platform features. The statistical drift test itself is code you own — most teams start from the Continuous Model Monitoring accelerator, built on open-source Evidently AI.

Three ways in, two ways to score

The demo used the first path because it is the fastest to show. Production teams typically use the second or third.

1

Direct from the UI

Pick a file and a function, size it, deploy.
Minutes to a working endpoint — ideal for iteration and for demos.

2

Through AI Registry

Promote a tracked experiment run into the registry, then deploy that version. Promotion becomes a governed, approvable event.

3

CI/CD via the SDK

A pipeline that has passed its KPI gate calls API v2 to trigger the job, create the build and promote the deployment — no human in the UI.

NOT EVERYTHING NEEDS AN ENDPOINT

For offline scoring, the same model runs as a batch Job on a schedule, is invoked through the SDK, or is orchestrated from Airflow in Cloudera Data Engineering. **One trained model, several consumption patterns — no rewrite between them.**

Where this goes next

The demo runs inside one workbench. Production deployments typically add these.



Registry-based promotion

Register the winning run into Cloudera AI Registry and deploy from there, so promotion is a governed, versioned, approvable event.



Production-scale serving

Cloudera AI Inference adds autoscaling, canary rollout and endpoint metrics over standard inference protocols.



Automated promotion

API v2 lets a CI pipeline that has passed the KPI gate trigger the job, create the build and promote the deployment with no human in the UI.



Dev, staging and production

Separate workbenches per environment against the same Data Lake, with promotion between them driven by Git and the API — the pattern most customers land on.

Why run the lifecycle here



Governance that is not bolted on

One identity model, one access policy and one lineage record across development, tracking, deployment and serving — because it is one platform, not four.



Bring the models to the data, not the data to the models

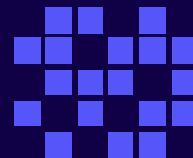
Models are built and served where the data already sits, under SDX. No export to a separate ML environment, and no third copy to secure and reconcile.



Open, so the exit stays cheap

MLflow tracking, standard inference protocols, open formats. Your team keeps familiar APIs and you keep the option to leave.

Hybrid by design — the same experience on public cloud, private cloud and on-premises.



Where we could take this

- **Run it on your data.** Swap the synthetic generator for a read against your own curated tables and re-run the identical pipeline.
- **Bring your platform team.** A second session on API v2, registry-based promotion and CI/CD integration.
- **Scope a pilot.** One real use case, one governed lifecycle, measured against your current time-to-production.